

HLW 0.9 save-architecture review

Full technical report | 29 September 2026
Source review: bf9fcd6f0bb5 | Repository references link to the reviewed commit.

1. Executive recommendation

I recommend a hybrid built around Option B: use sectors 30 and 31 as alternating copies of a required save extension, freeze the normal blocks at fixed capacities, and put all initialization and migration under one save manager.

This recommendation assumes the developers approve:

- Retaining **14 PC boxes**.
- Removing imported E-Reader Trainer Hill data and persistent recorded-battle playback.
- Establishing **0.9 as a new save baseline**, with pre-0.9 saves rejected explicitly.
- Supporting forward migration, without supporting downgrades.

The normal, ROM-defined Trainer Hill challenges can remain; repurposing its imported-data sector does not inherently require removing the facility.

The proposal below fits. I compiled both measurement probes and proposed-layout assertions using ARM GCC with the project's ARMv4T, Thumb, apcs-gnu, MODERN=1, TESTING=0 settings.

This was read-only. No repository files, generated files, ROMs, or saves were changed. Your two modified Safari Zone maps remain untouched. Compiler probes were created only under /tmp.

One important distinction from my previous answer: adding a separate flag bank can preserve today's saves. Your teammate's proposal deliberately takes a broader, final pre-0.9 format break. That is a product decision, not something required merely to add flags.

2. Verified inventory

Physical save blocks

The four preliminary measurements match the current compiler results exactly.

Persistent block	Compiled size	Physical capacity per save generation	Unused capacity
SaveBlock1	0x3D50 — 15,696	0x3E00 — 15,872	176
SaveBlock2	0x0F30 — 3,888	0x0F80 — 3,968	80
SaveBlock3	0x0640 — 1,600	0x0658 — 1,624	24
PokemonStorage	0x8560 — 34,144	0x8B80 — 35,712	1,568
Total	55,328	57,176	1,848

The capacities come from [save.h](#) and the sector assignments in [save.c](#).

Each normal sector contains:

- 3,968 bytes of its primary block.
- 116 bytes of SaveBlock3.
- A 12-byte footer.

There are 14 sectors per normal save slot.

RAM wrappers are not flash sizes

The ASLR wrappers add 128 bytes:

RAM allocation	Compiled size
SaveBlock1ASLR	0x3DD0 — 15,824
SaveBlock2ASLR	0x0FB0 — 4,016
PokemonStorageASLR	0x85E0 — 34,272

SaveBlock3 is not wrapped. These definitions are in [load_save.h](#).

Other verified structures and reservations

Structure or allocation	Bytes
BoxPokemon	80
Pokemon	100
HLWSaveExtension	512
RadioSaveData	152
AchievementSave	48
MiningWallSave	324
LegacyBag	880
BagExpansionSave	876
Bag expansion's internal reserve	132
Union Room reservation	210 , not 212
Enigma Berry reservation	52
NPCFollower	24
Roamer	28
HallofFameMon / team	24 / 144
Entire 30-team Hall of Fame	4,320
RecordedBattleSave	3,984
Trainer Hill header plus four floors	3,816

The Trainer Hill header alone is only eight bytes. Its existing size assertion therefore does not validate the full variable-length record.

All the supplied principal count measurements also matched, including:

- 318 flag bytes; 257 legacy variables.
- 1,524 species/form slots.
- 1,026 Pokédex slots; 129 bytes per dex bitmap.
- 838 items.
- 14 boxes; four fusion slots.
- One roamer; 128 berry trees.
- 80 mining attempt entries.
- 1,001 trainer IDs.

Dependencies that must become frozen ABI definitions

The block sizes depend on more than those headline counts:

Area	Dependencies requiring protection
SaveBlock1	Party and item-slot layouts; all bag capacities; flag/var counts; object events/templates; game stats; berries; secret bases; decorations; TV/news; mail/daycare; contest and record-mixing structures; roamers; dex/species bitmaps; mining; achievements; FREE_* switches.
SaveBlock2	Name lengths, option bitfields, dex layout, apprentices, minigame records, ranking records, Frontier party/mode/record counts and nested Pokémon representations; its FREE_* switches.
SaveBlock3	Fake RTC, NPC followers, first-time item descriptions, DexNav enablement and species count, Nuzlocke encounter-header count.
Storage	Box count, Pokémon size, box-name length, fusion count and party-Pokémon size.
Special records	Hall of Fame names/team count; recorded-battle action buffers and party layouts; Trainer Hill floor/trainer representations.

After 0.9, these should use fixed save-capacity constants or explicit wire structures. A feature being disabled must not remove its saved fields.

3. Current corruption risks

These are source-level findings. I did not reproduce them in an emulator or on hardware during this read-only review.

A. Confirmed shared-extension overlap

Offsets below are relative to `HLWSaveExtension.future[ ]`, which begins 160 bytes into the extension.

Bytes	Current ownership
0–15	Playlist 2 overflow
16–55	Playlist 3
56–63	Radio stickers
64–67	Shared Party/Summary theme — intentional
68–71	Pokédex theme and Wish Menu header
72–75	Battle speed and Wish Menu order
76–79	HP-bar option and Wish Menu order
80	Hungry Chansey marker and Wish Menu order
81–82	Remaining Chansey marker/version
83	Unused
84–339	64 Chansey records
340–351	Mining-session data

The Wish Menu writes all nine order bytes at 72–80, even when fewer actions are currently available. Overwriting Chansey's marker causes its next initialization to clear the request table.

Sources: [Wish Menu writes](#), [Chansey initialization](#).

B. Feature initialization can erase unrelated features

Radio, Party, Summary, Pokédex, Options, and Chansey code independently clear the entire 512-byte extension on shared-header mismatch.

A shared version bump can therefore erase unrelated settings and progress. The save manager must own this operation. Features should never reset the whole extension themselves.

C. SaveBlock3 is not checksummed

The current sector checksum covers the primary block data, excluding the appended 116-byte SaveBlock3 stripe.

That leaves RTC, follower, DexNav, and Nuzlocke data without the normal checksum protection. See [sector writing](#) and [stripe copying](#).

D. Partial saves leave most of SaveBlock3 unchanged

Several save paths rewrite only logical sectors 0–4. That saves only:

$$5 \times 116 = 580 \text{ bytes}$$

of the current 1,600-byte SaveBlock3. Its remaining 1,020 bytes retain older values.

This affects the architecture of link, Frontier, contest, record-mixing, and related saves—not just the ordinary Save menu.

E. Loader validation is insufficient

The current loader:

- Indexes its sector-location table using a flash-provided ID before validating its bounds.
- Does not require all sectors in a slot to have the same save counter.
- Can copy data and invoke migration after an empty or unsuccessful load.

These need correction before claiming corruption-safe migration. See [save-slot loading and validation](#).

F. Bag clearing uses an unsafe transfer length

CpuFastFill is used on the 876-byte expansion and 880-byte legacy bag. BIOS FastSet operates in eight-word chunks; these lengths are not multiples of 32 bytes. The expected overruns are:

- Expansion: 20 bytes into the following dex data.
- Legacy bag: 16 bytes into following Pokéblock data.

This needs targeted runtime confirmation and replacement with an appropriately sized fill. Sources: [bag clearing](#), [BIOS interface documentation](#).

Separately, MigrateBagExpansion treats any unknown version/size as permission to clear the expansion. That is unsuitable for post-0.9 migration.

G. Achievement and Nuzlocke bookkeeping already have problems

- Wish-form tracking occupies achievement bits 30–124. Adding achievements shifts its starting point.
- Its preprocessor overflow check uses an enum constant, which the preprocessor evaluates as zero.
- Its “last 95 species” calculation currently starts at Iron Boulder, not an explicit list of HLW Wish forms.
- Nuzlocke reserves 139 encounter-header entries, while the current generated table contains 151 non-sentinel entries, representing 142 distinct maps. Out-of-range entries are treated as unset and cannot record their restriction state.

Sources: [achievement definitions](#), [Wish tracking](#), [Nuzlocke capacity](#).

H. Saved ROM pointers threaten cross-build compatibility

NPCFollower.script persists a raw ROM address. Relinking a later ROM can change the script’s address without changing the save’s size.

Replace it with a stable script ID or reconstructable NPC identity. Ordinary object-template scripts are refreshed on continue, but that refresh uses template order, which also needs stable identity handling.

Findings that should not be overstated

- The special-sector loops do **not** run past the full 4,096-byte staging buffer: they copy 4,092 bytes beginning at offset four. Their comments and unbounded API are misleading, but the suspected whole-buffer overflow is not present.
- The SB1/SB2 sizes in src/data/trade.h belong to an explicitly unused table.
- Hall of Fame, secret bases, and Frontier records do not all store full 80/100-byte Pokémon. They have separate representations.

Stale measurements in comments/tests

Examples of actual offsets:

Field	Actual offset
SaveBlock1.flags	0x1294
SaveBlock1.vars	0x13D2
SaveBlock1.hlwSave	0x3B50
Storage boxes	0x0004
Storage fusions	0x83D0

The existing [save tests](#) also contain obsolete SB1/SB2/SB3 sizes. They need replacement with a real ABI manifest.

4. Logical-ID risk register

Freezing offsets alone will not preserve meaning.

Saved namespace	Main risk	Required rule
Species/forms	Enum insertion changes Pokémon, eggs, encounters, and tracking	Explicit stable IDs; never recycle retired IDs.
Moves	Reordering changes saved moves and facility records	Append-only IDs; preserve tombstones.

Saved namespace	Main risk	Required rule
Items/balls	Changed IDs reinterpret inventory and held items	Stable IDs and stable ball encoding.
Trainers	Trainer count currently moves system flags	Separate runtime count from fixed defeated-trainer storage.
Flags/vars	New ranges overlap or grow old arrays	Fixed ranges, bounded routing, permanent ownership.
Maps/warps	Generated group/order changes relocate saved players	Stable map IDs; migration for changed geometry/warp meaning.
Local NPCs/scripts	Reordered objects or raw pointers change behavior	Stable local/script IDs; reconstruct ROM pointers.
Game stats	Index insertion changes counters	Explicit immutable indexes.
Achievements/counters/Wish forms	Shared bit positions and changing list order	Independent fixed namespaces and bitmaps.
Mining walls/locations	Coordinates or enum order change identity	Stable wall/location IDs.
Hidden grottos	Table order and packed species/item IDs change meaning	Stable grotto IDs and documented packing.
Wild encounter headers	Generated table position is saved	Dedicated stable encounter IDs.
Radio	Playlist entries depend on song numbering	Permanent song IDs; removed songs retain tombstones.
UI themes/Wish actions	Enum order changes saved selection/order	Stable action/theme IDs; merge new actions into old orders.
Decorations/wallpapers	Reordering reinterprets saved room/box choices	Stable IDs and safe fallback for retired content.
Berry, mail, facility, record-mixing data	Compact IDs and nested layouts change	Frozen representations plus independent validation.

Map compatibility also needs attention beyond numbering: if an update removes a doorway or changes a saved position into a wall, a content migration must relocate the player safely.

5. Architecture comparison

Option	Available space and flexibility	Main tradeoff	Assessment
A: Normal blocks only	1,848 tail bytes; 2,242 pooled bytes when current Union/Enigma/bag reservations are included	Tight allocation, substantial repacking; preserves special features	Feasible, but limited long-term reserve.
B: Mirrored sectors 30/31	Adds 4,032 usable bytes , not 8 KiB	Removes imported E-Reader/replay storage; requires transactional integration	Best foundation if those features are dispensable.
C: Extended boxes	Space depends on how many special sectors and core reservations are surrendered	Competes directly with postgame/system storage; highest regression exposure	Choose only for a concrete box-capacity requirement.
D: Hybrid	Fixed normal blocks plus deliberate relocation and optional auxiliary storage	More ownership/accessor work, but less waste	Recommended allocation strategy combined with B.

Option A is possible

It would be incorrect to say the requested expansions cannot fit normal blocks.

A representative capacity expansion consumes about 2,042 of the 2,242 pooled bytes, leaving only 200 before additional metadata and other growth. Compacting mining into stable-ID bitmaps improves this to approximately 648.

These are aggregate budgets: an actual design must still solve alignment and distribution across physical blocks.

Runtime, wear, and implementation costs

- B adds one sector to a normal save: **15 instead of 14**, a 7.1% increase in programmed-sector count, plus verification work.
- Existing five-sector partial saves would become full transactions, so their latency increase is larger.
- Each extension copy is erased approximately once every two saves.
- One resident extension requires 4,032 payload bytes, or 4,096 including its header—not two RAM copies.
- Existing sector scratch storage can be reused.
- A requires no extra normal-save sector, but still needs loader, checksum, migration, and partial-save corrections.
- C consumes much of the same flash budget that B uses for system growth.

SoulGold comparison

SoulGold demonstrates useful patterns: guarded flag ranges, explicit storage checkpoints, required-extension validation, and forward-only migrations. Its release notes confirm the 16-to-19-box transition. [Save implementation](#), [release notes](#).

Its capacity figures cannot be transplanted:

- Boxed Pokémon are **76 bytes**, versus HLW's 80.
- It uses all four special sectors for mirrored overflow/auxiliary records.
- Its SB1 also carries a 1,944-byte storage tail.
- Its current SB1 future reserve is only 34 bytes.

Sources: [storage layout](#), [global layout](#).

In HLW, a contiguous fifteenth box adds **2,412 bytes**, leaving an 844-byte shortage against today's storage tail. A box expansion would therefore need its own allocation decision.

I reviewed SoulGold's public branch, not a successfully pinned commit. Its reported testing is useful evidence, not proof of exhaustive interrupted-save safety.

6. Proposed byte-level layout

Everything in this section is a **design proposal**. The component sizes below passed ARM compiler assertions; integration and behavior have not been implemented or tested.

A. Physical ownership

Sectors	Proposed owner
0–13	Normal slot A
14–27	Normal slot B
28–29	Alternating Hall of Fame archive banks
30	Required extension for slot A
31	Required extension for slot B

Pad the normal structures to their existing physical maxima:

```
SaveBlock1    == 15872;
SaveBlock2    == 3968;
SaveBlock3    == 1624;
PokemonStorage == 35712;
```

This adds 1,848 RAM bytes but makes accidental append-only growth fail immediately. Future features must consume named reserve, not enlarge the structures.

Keep the existing SB1/SB2 field positions where practical. Fields moved elsewhere become explicitly retired reservations, not duplicate live data.

B. Extension-sector envelope

A fixed 64-byte envelope leaves 4,032 payload bytes:

Offset	Bytes	Meaning
0x00	4	New format magic
0x04	2	Physical envelope version
0x06	2	Header length: 64
0x08	2	Public save schema
0x0A	2	Payload length: 4,032
0x0C	4	Save generation
0x10	16	Persistent file UUID

Offset	Bytes	Meaning
0x20	4	Main-image CRC32
0x24	4	Extension-payload CRC32
0x28	4	Referenced Hall of Fame generation
0x2C	4	Referenced Hall of Fame CRC32
0x30	1	Normal slot
0x31	1	Hall of Fame bank, or none
0x32	2	Defined flags
0x34	4	Header CRC32
0x38	4	Reserved
0x3C	4	Extension-complete marker

Use explicitly specified little-endian encoding and CRC parameters. The main digest covers every fixed main-block byte, including all SB3 stripes and reservations.

The UUID/schema also exist at a frozen location in the normal save. Trainer ID alone is insufficient for pairing files.

C. HLWSaveBlock4 payload

```
struct HlWSaveBlock4Payload
{
    u8 customFlags[256];      // 2,048 flags
    u16 customVars[256];      // 512 bytes
    u8 dexNavSearch[2048];    // Stable species/form slots
    u16 grottoStates[64];     // 128 bytes
    u8 futureReserved[1088];
};                          // Exactly 4,032 bytes
```

Payload offsets:

Offset	Bytes	Owner
0x000	256	Custom flags
0x100	512	Custom vars
0x300	2,048	DexNav
0xB00	128	Hidden grottos
0xB80	1,088	Unallocated reserve

Proposed ID ranges:

- Custom flags: 0x1000–0x17FF.
- Trainer-defeated flags: 0x2000–0x27FF.
- Custom vars: 0x5000–0x50FF.

Keep current legacy flags and vars separately fixed. Both dispatchers must validate bounds and reject undefined gaps.

D. Rebuilt SaveBlock3

Moving DexNav out creates room for fixed-capacity shared systems:

Offset	Bytes	Owner
0x000	16	Manager-owned metadata
0x010	12	Fake RTC
0x01C	24	NPC follower with stable script ID
0x034	4	Party follower, DexNav chain, reserved
0x038	256	2,048 defeated-trainer bits
0x138	192	1,536 dex-seen bits
0x1F8	192	1,536 dex-caught bits
0x2B8	256	2,048 released-species bits
0x3B8	128	512 encounter IDs, two bits each
0x438	192	Achievements
0x4F8	68	Mining
0x53C	256	Reserved first-time item tracking
0x63C	28	Reserve
End	1,624	

The item bitmap is deliberately overprovisioned. It does not increase Pokémon’s current held-item encoding limit.

Achievements receive:

- 256 independent achievement bits.
- 128 independent Wish-form bits.
- 32 full u32 counters.
- A participation mask, queue, and metadata.

Mining receives:

- A 256-wall daily attempted bitmap.
- 32 location counters.
- Day and reserved fields.

The mining redesign requires a stable wall-ID table. The current implementation records coordinate keys, so simply changing its array count would not achieve this layout.

E. Corrected 512-byte media extension

Offset	Bytes	Owner
0	8	Manager-owned header
8	208	Radio, playlists, favorites, stickers
216	4	Shared Party/Summary theme, dex theme, battle speed, HP style
220	12	Wish Menu count/order/reserve
232	256	64 Chansey records
488	24	Reserve

This preserves the current capabilities without overlapping ownership.

F. Pokémon Storage tail

The original storage core remains 34,144 bytes. Allocate its 1,568-byte tail as follows:

Offset	Bytes	Owner
0x8560	64	Global schema/UUID metadata
0x85A0	288	Two Hall of Fame teams
0x86C0	400	Four additional reserved fusion slots
0x8850	196	Seven additional reserved roamers
0x8914	336	Bag supplement
0x8A64	284	Reserve
End	35,712	

That reserves capacity for eight fusions and eight roamers while allowing current gameplay to use fewer.

G. Bag capacities and fixed segmentation

Recommended capacities:

Pocket	Slots
Items	160
Medicine	64
Key items	64
Poké Balls	32
TMs/HMs	128

Pocket	Slots
Berries	72
Total	520

These consume 2,080 slot bytes.

An exact allocation fits the existing legacy bag, existing 876-byte expansion, and proposed 336-byte supplement:

- Legacy bag: retain its 880-byte physical region.
- Expansion: header 8; extra Items 384; Medicine 256; extra Balls 64; first extra TM segment 164.
- Supplement: remaining TMs 92; extra Keys 136; extra Berries 104; reserve 4.

All pocket segments get named, fixed offsets. Changing an active capacity later must not move another pocket.

H. Hall of Fame safety

Preserve all 30 teams:

- Each archive sector: 64-byte header plus 28 teams × 144 bytes = 4,096.
- Remaining two teams: 288 bytes in the normal save.

Hall of Fame banks alternate independently of normal-slot parity. Ordinary saves retain the current archive reference; entering the Hall of Fame prepares the inactive archive before committing the new main save.

Reserve remaining

Besides the reserves explicitly shown above, SB1 gains reusable retired regions when dex tracking, mining, and achievements move: **821 bytes**, plus its existing 210-byte Union Room reservation and 52-byte Enigma reservation.

These are separate owned regions, not one contiguous allocation. The manifest must record them individually.

7. Migration and transaction lifecycle

Loading

1. Scan both normal slots without copying their contents into live game state.
2. Validate sector IDs before indexing any tables.
3. Require one complete set of logical sectors with a consistent generation.
4. Locate the extension assigned to that physical slot.
5. Validate UUID, generation, schema, lengths, header CRC, payload CRC, and complete main-image CRC.
6. Validate any required Hall of Fame reference.
7. Select the newest complete supported bundle.
8. Load it into RAM.
9. Run sequential migrations.
10. Only then load party/bag caches, reconstruct pointers, and initialize features.

A complete save with a newer unsupported schema must produce an explicit “newer version required” result. Do not silently select an older generation and allow it to overwrite newer progress.

A valid main slot with a missing required extension is an incomplete bundle. Do not manufacture a default extension.

Migration

Use one authoritative interface:

```
enum SaveMigrationResult ValidateAndMigrateSave(void);
```

Requirements:

- Public 0.9 begins at schema 1, distinct from development-only tags.
- Migrations proceed through explicit version steps.
- A step advances its version only after succeeding.
- Migration changes RAM only.
- Existing flash remains untouched until the next normal save.
- Failure blocks gameplay/saving instead of resetting unrelated data.
- No feature independently clears a shared block.

Boot currently loads saves before initializing the heap, so migration must use bounded static scratch or deliberately change that initialization order.

Encrypted money, quantities, and stats must be interpreted under the loaded key. Any new encrypted pockets must participate in both cache copying and rekeying.

Saving

1. Identify the currently valid bundle and preserve it.
2. Freeze the intended saved state, including bag/party/object caches.
3. Select the inactive normal slot and its extension sector.
4. Prepare and verify a new Hall of Fame bank if needed.
5. Write and verify the target extension with the next generation.
6. Write the target normal sectors, withholding the final commit signature byte.
7. Read back and validate the complete prepared bundle.
8. Program and verify the commit byte last.
9. Update the in-memory committed-generation state.

All normal, link, Frontier, contest, overwrite, and other save paths must use this transaction. Incremental scheduling can remain for link timing, but in-place partial saves cannot remain.

Interruption	Expected recovery
During RAM migration	Reload the unchanged old save and migrate again.
During extension preparation	Previous complete bundle remains valid.
During normal-sector writing	Incomplete target is rejected; previous bundle loads.
After successful final commit	New complete bundle loads.
Ambiguous final-write error	Rescan flash before retrying.
Newest extension corrupt	Recover the previous complete matching bundle and notify the player.

New-game overwrite must not erase the old required extension before the replacement commits. Preserve physical generation ordering while assigning the new file a fresh UUID.

8. Frozen capacities and Pokémon boundaries

System	Recommended capacity	Saved byte cost
Trainers	2,048	256
Custom flags	2,048	256
Custom vars	256	512
Species/form tracking	2,048	Depends on field; released bitmap 256
National Dex tracking	1,536	192 per bitmap
Distinct DexNav slots	2,048	2,048
Encounter IDs	512	128
Achievements	256	32-bitset bytes
Wish forms	128	16
Achievement counters	32	128
Mining walls / locations	256 / 32	68 combined
Hidden grottos	64	128
PC boxes	14	420 boxed Pokémon
Fusion capacity	8 reserved	800
Roamer capacity	8 reserved	224
Chansey records	64	256
Berry trees	128 initially	1,024

For trainers, 1,536 slots cost 192 bytes and leave 535 IDs beyond the current count. A 2,048-slot bank costs 64 more bytes and leaves 1,047. I recommend the latter.

DexNav alternatives

- **2,048 distinct stable slots:** preserves form-specific progression; recommended.
- **1,536 National Dex slots:** saves 512 bytes but merges form progression.
- **Direct runtime species indexes:** acceptable only if those IDs are permanently stable and capacity is fixed.

If merging forms, migrate by an explicit rule such as maximum search level, not an accidental sum.

Freeze Pokémon serialization

Keep BoxPokemon == 80 and Pokemon == 100.

There are 22 candidate unused bits: 20 in checksum-protected substructures and two in the header. Reserve and initialize them centrally; do not assume every unused-looking ribbon or external-tool bit is available.

Current encodings impose additional limits:

- Species and moves: 11 bits.
- Held items: 10 bits.
- Ball IDs and other compact fields have their own limits.

A larger tracking array does not increase these limits. Future use of reserved bits requires a defined codec/migration change and serialized-byte tests.

A per-Pokémon sidecar is possible but expensive and operationally complex. Even two bytes per currently persistent owned-Pokémon position cost 864 bytes. It must follow every deposit, trade, daycare operation, fusion, release, and rearrangement. Prefer derived data or explicitly reserved bits first.

Link trade already transmits raw Pokémon records. Record mixing transmits nested record structures. Default to a checked HLW protocol version; vanilla/older-HLW compatibility requires deliberate translation.

9. Implementation sequence

1. **Audit tooling and assertions:** capture sizes, every persisted field offset, bit encodings, IDs, and configuration.
2. **Correct current defects:** overlap, unsafe clearing, loader bounds, checksum gaps, Nuzlocke capacity, and broken guards.
3. **Define stable capacities and IDs:** decouple ROM content counts from storage capacities.
4. **Introduce named persistent structures:** route access through shared helpers.
5. **Introduce the centralized schema manager:** explicit new-game, unsupported, corrupt, and migrated states.
6. **Implement the complete transaction:** extension pairing, full saves, link barriers, failure handling, and Hall of Fame.
7. **Port existing features without behavioral redesign.**
8. **Add fixtures and fault-injection tests.**
9. **Freeze and document public schema 1.**

Intermediate development commits may be save-incompatible. Do not designate any as the permanent 0.9 baseline until the transaction and fixtures pass.

Pre-0.9 data should be rejected clearly, not silently reinterpreted or automatically erased. A personal-save converter can be a separate, optional project.

10. Tests and ongoing governance

Required tests

- Exact sizes, offsets, alignment, ranges, capacities, and reserved-bit encodings.
- New game, repeated load/save, reset, overwrite, and first-ever save.
- Every save entry point, including trade and old partial-save callers.
- Early/mid/postgame fixtures with populated features.
- Full bags, boxes, fusions, roamers, achievements, playlists, mining, and grotto state.
- Encryption-key changes and bag-cache synchronization.
- All published-schema migrations, including repeated migration attempts.
- Corruption of IDs, counters, schema, lengths, UUIDs, CRCs, SB3 stripes, and archive references.

- Power interruption at each erase/program/verify/commit stage.
- Recovery with only one valid bundle.
- Unsupported newer saves and pre-0.9 saves.
- Real hardware and emulator tests using battery saves; save states are not a compatibility substitute.
- Heap and link-timeout measurements under the slowest save paths.

Padding and a resident 4,096-byte extension add approximately **5,944 static RAM bytes**. The existing ELF has about 12,868 bytes of static address-space slack, leaving an estimated 6,924 before new manager state. That is an existing-build estimate, not a successful link or runtime-memory validation of the new design.

Repository policy

Check in:

- `docs/save_abi.md`.
- A machine-readable ABI/ID/ownership manifest.
- Exact compile-time assertions.
- Golden 0.9 saves and expected decoded values.
- Migration and interrupted-save tests.

CI should fail when an existing field moves, an ID changes meaning, a capacity is exceeded, or a feature claims already-owned reserve.

Upstream merges must adapt to HLW's save format. They must not redefine it through changed counts, configuration switches, bitfields, or generated ordering.

Keep the existing external ROM-header interface stable where practical and publish a separate HLW descriptor for split pockets and extension blocks. The vanilla-style header cannot fully describe this format.

11. Decisions needed before implementation

The smallest useful decision set is:

1. **Save policy:** confirm that pre-0.9 saves may be rejected and downgrades are unsupported. Is a separate converter desired for your personal testing save?
2. **Special features:** may imported E-Reader data and recorded-battle playback be removed? My recommendation assumes yes.
3. **Pokémon storage:** is 14 boxes acceptable as the supported capacity? I recommend reserving eight fusion slots, even if only four are initially active.
4. **Growth profile:** approve the capacity table—especially 2,048 trainers/forms/DexNav slots, distinct form progression, 256 custom vars, 256 achievements, and 256 mining walls—or identify expected growth beyond it.
5. **Interoperability:** must vanilla/older-HLW trades or existing external tools work unchanged? My default is version-checked HLW links and a documented HLW tool interface.

Bottom line: your teammate's preferred direction is sound, but it needs a coordinated save transaction and ownership redesign. Adding sectors without fixing checksums, partial saves, shared resets, and ID stability would leave substantial corruption risks in place.